# Estándar de Desarrollo — Proyectos Hospital de Ovalle

Oct 13, 2026

## Backend (`<proyecto>-api`)

- **Node.js + Express + TypeScript**
- Estructura por capas (no por módulo de dominio): `config/ controllers/ routes/ services/ middlewares/ repositories/ interfaces/ helpers/ prisma/`
- ORM: **Prisma** sobre MySQL
- Validación de entrada: **zod**
- Documentación de API: **Swagger** (swagger-jsdoc + swagger-ui-express) en `/api/docs`
- Seguridad base: helmet, cors (origen restringido a `FRONTEND_URL`), express-rate-limit en rutas sensibles
- Autenticación: **Keycloak OIDC** — el backend es el cliente confidencial; el frontend nunca maneja tokens
- Datos de pacientes/funcionarios: siempre vía API Pacientes externa, nunca consulta directa a la BD hospitalaria

### Decisiones ya tomadas (no se vuelven a discutir salvo que cambien)

| Decisión | Estándar |
|---|---|
| ORM | Prisma (no mysql2 crudo, Drizzle o TypeORM salvo NestJS explícito) |
| Estructura | Por capas, no por dominio |
| Auth | Keycloak SSO — reemplaza JWT+bcrypt propio y 2FA manual |
| Docs de API | Swagger siempre incluido |
| Datos de pacientes | Siempre vía API Pacientes externa |

## Frontend (`<proyecto>`)

- **Angular** (última versión **estable**), standalone components (`app.config.ts` + `app.routes.ts`, sin NgModules)
- UI: **PrimeNG + Tailwind CSS** (sin Angular Material salvo petición explícita)
- Estructura: `core/` (guards, interceptors, servicios singleton, modelos), `shared/` (componentes/pipes/directivas reutilizables), `features/<dominio>/`
- `AuthService` solo redirige al backend (`/auth/login`, `/auth/logout`) y consulta `/auth/me` — sin lógica OIDC propia

## Infraestructura común

- Dos repos separados por proyecto: `<nombre>` (frontend) y `<nombre>-api` (backend)
- Dockerfile + Jenkinsfile (pipeline estándar del hospital) en ambos repos
- Frontend: Dockerfile multi-stage, build de Angular servido por nginx
- `.env.example` con todas las variables usadas, en ambos repos
- README con instrucciones de instalación/arranque local
- Socket.io solo si el módulo necesita tiempo real — no se incluye por defecto

### Auth: Keycloak (SSO)

- Backend = cliente confidencial OIDC (Authorization Code + PKCE S256), valida contra JWKS del realm con `jose`
- Endpoints: `GET /api/auth/login`, `GET /api/auth/callback`, `GET /api/auth/me`, `POST /api/auth/refresh`, `POST /api/auth/logout`, `POST /api/auth/backchannel-logout`
- Roles de Keycloak mapeados a `role_id` local en MySQL (tabla `users` con `keycloak_subject` único); auto-registro por email/username en el primer login
- Tabla `revoked_sessions` para Backchannel Logout en despliegues multipod
- No se implementa login propio ni 2FA manual (speakeasy/qrcode) salvo indicación explícita en contrario

## Alternativa de backend: PHP (Laravel)

Si se reemplazara el backend actual, Laravel es el equivalente más directo a Node + Express + Prisma:

| Pieza actual | Equivalente en Laravel |
|---|---|
| Express + TypeScript | Laravel (PHP 8+) |
| Prisma | Eloquent ORM |
| zod | Form Requests / Validator |
| Swagger (swagger-jsdoc) | L5-Swagger / Scramble |
| Keycloak vía `jose` + JWKS | Socialite + OIDC, o `php-jwt` contra JWKS |
| Estructura por capas a medida | `app/Http/Controllers`, `app/Models`, `app/Services`, `routes/api.php` (ya vienen separados) |

**Trade-off**: migrar a PHP rompe el lenguaje compartido entre frontend y backend (hoy un desarrollador se mueve entre Angular y la API sin cambiar de lenguaje). A favor de PHP: hosting más barato en VPS compartidos, más disponibilidad de desarrolladores PHP en el mercado local, y Laravel trae de fábrica cosas que hoy se configuran a mano (migraciones, colas, validación). Con Prisma y Keycloak ya estandarizados en ~15 proyectos, no hay una razón técnica fuerte para migrar — solo tendría sentido si el equipo que mantiene el código cambia a personas con background PHP.

## Otras alternativas de backend

| Alternativa | Cuándo tendría sentido |
|---|---|
| .NET (C#) | Si hay sinergia con infraestructura Windows/IIS del hospital |
| Java (Spring Boot) | Si se busca un perfil más "enterprise" o integración con sistemas Java existentes |

Ninguna de las dos ofrece mejor fit que PHP frente al stack actual; se incluyen solo como referencia de mercado.